Mitali Chugh

nexusreport_5th may

Quick Submit

Quick Submit

UPES: University of Tomorrow

Document Details

Submission ID

trn:oid:::1:3560290358

Submission Date

May 5, 2026, 9:57 PM GMT+5:30

Download Date

May 5, 2026, 10:03 PM GMT+5:30

File Name

Sutrava_Project_Report_F.pdf

File Size

3.6 MB

38 Pages

11,870 Words

70,855 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

"Sutrava: An AI-Driven System for Automated Requirements Engineering from Unstructured Stakeholder Communication"

A

Project Report

submitted in partial fulfillment of the
requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE & ENGINEERING

by

NAME	Roll No.
Devanshi Jain	R2142230278
Shivang Mangal	R2142230269
Shreyash Shivhare	R2142231555
Sreyas Sharma	R2142230271

under the guidance of

Dr. Mitali Chugh

School of Computer Science

University of Petroleum & Energy Studies

Bidholi, Via Prem Nagar, Dehradun, Uttarakhand

May – 2026

CANDIDATE'S DECLARATION

We hereby certify that the project work entitled "Sutrava: An AI-Driven System for Automated Requirements Engineering from Unstructured Stakeholder Communication" in partial fulfilment of the requirements for the award of the Degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING with specialization in DEVOPS and submitted to the Department of Systemics, School of Computer Science, University of Petroleum & Energy Studies, Dehradun, is an authentic record of our work carried out during a period from **January, 2026 to May, 2026 under the supervision of Dr. Mitali Chugh, Senior Associate Professor, Program Lead DevOps**.

The matter presented in this project has not been submitted by me/ us for the award of any other degree of this or any other University.

Devanshi Jain	**R2142230278**
Shivang Mangal	**R2142230269**
Shreyash Shivhare	**R2142231555**
Sreyas Sharma	**R2142230271**

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date: 06.05.2026

Dr. Mitali Chugh

Project Guide

ACKNOWLEDGEMENT

We wish to express our deep gratitude to our guide **Dr. Mitali Chugh**, for all advice, encouragement and constant support she has given us throughout our project work. This work would not have been possible without his support and valuable suggestions.

We sincerely thanks to our respected **Dr. Christalin Nelson, Cluster Head CSO,** for his great support in doing our project in **Sutrava: An AI-Driven System for Automated Requirements Engineering from Unstructured Stakeholder Communication.**

We are also grateful to Dean SoCS UPES for giving us the necessary facilities to carry out our project work successfully. We also thanks to our Course Coordinator, Dr. Riya Sharma and our Activity Coordinator Dr. Vikramjit Singh Bhathal for providing timely support and information during the completion of this project.

We would like to thank all our **friends** for their help and constructive criticism during our project work. Finally, we have no words to express our sincere gratitude to our **parents** who have shown us this world and for every support they have given us.

Name	**Devanshi Jain**	**Shivang Mangal**	**Shreyash Shivhare**	**Sreyas Sharma**
Roll No.	**R2142230278**	**R2142230269**	**R2142231555**	**R2142230271**

ABSTRACT

Requirements engineering has traditionally been among the most challenging aspects of software development, which may seem counterintuitive at first sight, since it would appear to be relatively easy in theory. In practice, however, requirements elicitation and management turn out to be surprisingly error-prone and tedious tasks due to the inherently informal nature of stakeholder communication (usually, via email, Slack chat rooms, Jira comments, and meetings). In fact, extracting, organizing, and prioritizing software requirements from various communication channels remains a significant challenge for modern software project managers. The present project introduces an innovative artificial intelligence system called SUTRAVA, which solves precisely this problem by leveraging NLP, deep learning, and rule-based techniques to implement a holistic requirements engineering pipeline.

The proposed solution relies on raw text input provided by stakeholders and applies eight consecutive processing stages. First, sentences are detected, and requirement statements are identified using a fine-tuned binary DistilBERT classifier. Next, the identified sentences go through a pre-trained BERT-based Named Entity Recognition (NER) module based on spaCy 3, which recognizes five types of entities: ACTOR, FEATURE, QUALITY_ATTRIBUTE, CONDITION, and PRIORITY_INDICATOR. Entity extraction is followed by arranging the entities in the actor-action-constraint schema, performing clustering using the Sentence-BERT embedding combined with UMAP and HDBSCAN techniques, and entity ranking through a multisignal scoring engine involving keyword-based analysis, sentiment-based analysis, semantic corrections, decision-making rules, and an optional LLM-based evaluation process. Afterward, the clusters will be summarized using the T5 transformer model, and the output is recorded in a structured form in JSON and Markdown files, as well as the console logs and explainability narratives.
SUTRAVA’s application architecture is designed to run on the desktop as a client built using Electron and React frameworks together with Spring Boot RESTful service API that receives input from the Atlassian Jira API and Slack API using OAuth2 protocol.The user interface provides visual dashboards where users may explore extracted requirement sentences, their classification and clustering results and assigned priority levels.

Synthetic and semi-real datasets were used to evaluate the performance of the proposed approach. DistilBERT demonstrated high accuracy in identifying requirements. Although the entity extraction capabilities of the BERT-NER module were rather moderate, given the limited amount of training data available to us, the multi-signal prioritization engine generated reasonable priority scores. Moreover, semantic correction significantly decreased the number of HIGH false positive detections. Overall, SUTRAVA proves that the combination of transformer-based NLP and specific domain knowledge can substantially reduce manual requirements engineering labor.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

By definition, requirements engineering implies an assessment of requirements for a software system prior to development. This practice includes requirement elicitation, analysis, specification, and validation processes which despite seeming quite straightforward happen to be the most labor-intensive stage of development. Studies show that more than half of projects' failures result from poorly understood, incomplete, and ambiguously formulated requirements.

In today's software industry with its frequent use of agile development methods, communication with stakeholders does not follow the formal process anymore. Instead, product owners discuss new features via Slack messages, clients voice their concerns via emails, developers submit bug tickets as short comments on Jira, and vital decisions are made during meetings that are never formally reviewed. In sum, you get an inconsistent bunch of communication artifacts containing a significant amount of noise along with valuable requirements.

Traditionally, the task of identifying and documenting requirements lies with the business analyst or project manager who must read all these artifacts and extract requirements in a highly manual way. However, it is time-consuming and error-prone. The result is missing requirements, incorrect priorities assignment, and increasing gap between the actual desires of stakeholders and the final product that the development team created.

In recent years, significant progress has been achieved in such fields as NLP and transformers. Various pre-trained language models such as BERT, DistilBERT, and GPT show a remarkable performance when it comes to understanding the meaning conveyed by the text. Moreover, these models can be fine-tuned for various downstream tasks including binary classification (is the sentence a requirement or not), named entity recognition (entities within requirement), or ranking (importance of each request).

Based on these advancements, we developed SUTRAVA—a full-cycle NLP-powered solution for automated extraction, structuring, clustering, prioritizing, and summarizing of software requirements derived from unstructured data containing communication between developers and other stakeholders. The purpose of our work is to facilitate the routine part of this work by automating the whole process and leaving the analyst's role for human validation and negotiations.

1.2 Problem Statement

Requirements extraction from the stakeholders' communication in an unstructured form such as emails, chat messages, comments in the ticketing tool, and meeting notes is time-consuming, inaccurate, and highly subjective. This calls for a smarter solution that would be able to automatically identify sentences that contain requirements, extract information from them in a structured manner (actors, features, constraints, quality attributes), categorize and prioritize them using different criteria, cluster similar sentences, and create an output that could be traced and explained while also integrating into the existing development process toolset.

1.3 Objectives

1.3.1 Main Objective

The overall aim would be to develop an artificial intelligence based system to automate the process of requirements engineering. This system should be capable of taking input from the unstructured data generated by stakeholders' communications and then using natural language processing techniques along with machine learning algorithms for detecting, structuring, clustering, prioritizing, and summarizing software requirements.

1.3.2 Sub Objectives

● Designing an efficient binary requirement classifier based on fine-tuned DistilBERT, which can reliably detect sentences containing requirements from general conversations.
● Creation of the Named Entity Recognition tool using spaCy 3 and transformers powered by BERT that can detect and extract domain-specific entities from the requirement text (ACTOR, FEATURE, QUALITY_ATTRIBUTE, CONDITION, PRIORITY_INDICATOR).
● Development of the requirement structuring tool that transforms the requirement text, enhanced with NER results, into the actor-action-constraint form and categorizes them into functional and non-functional requirements.
● Creation of the semantic clustering pipeline with Sentence-BERT embeddings, UMAP, and HDBSCAN algorithms used for the automatic clustering of semantically similar requirements.
● Design of the multi-level prioritization engine that performs keyword identification, sentiment analysis, semantic correction, rule-based arbitration, and optional auditing using large language models (LLMs).
● Implementation of abstractive summarization with T5 and the transparency engine that generates an elaborate explanation for each step taken at the stage of processing the requirement text.
● Development of cross-platform software with Electron and React front-end, Spring Boot back-end, and integration with external systems such as Jira and Slack using OAuth2 authentication.
● Verification of the implemented system against general NLP metrics (precision, recall, accuracy, F1-score), and its practical implementation illustrated with the example of stakeholder communication data.

1.4 Project Timeline (PERT Chart)

Our project implementation involved two phases that were completed within two separate semesters. Phase one included research, literature analysis, modeling, and design of the NLP pipeline. In contrast, the second stage involved developing the full stack application.

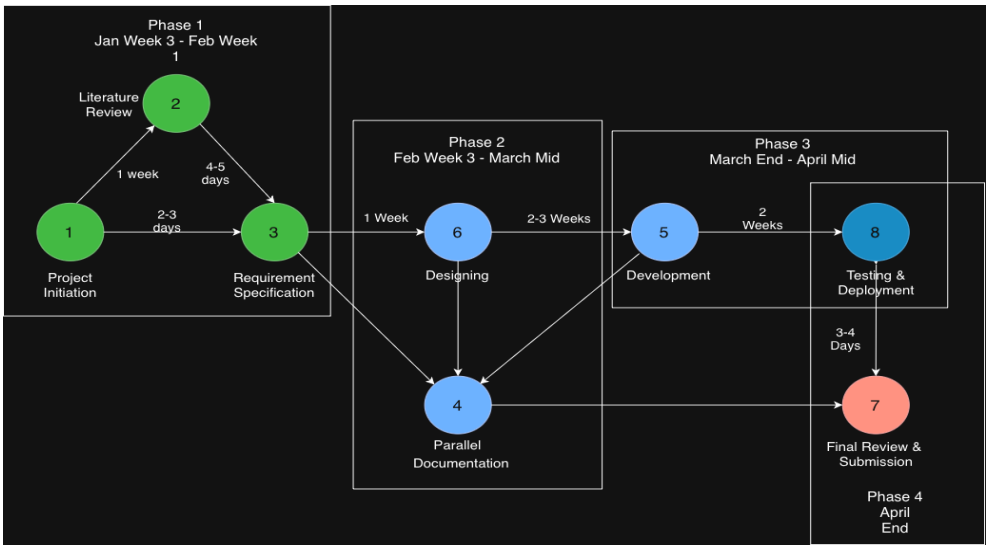

Fig. 1.1: PERT Chart for Project Timeline — (Refer to PERT CHART.drawio.png in the project repository)

The major milestones in our implementation were:

- Month 1–2: Literature survey, studying relevant papers and datasets, designing the annotation process.
- Month 3–4: Building the DistilBERT classifier, building the NER model using spaCy+BERT, training and testing the entire pipeline.
- Month 5–6: Clustering, prioritizing, semantically correcting and summarizing the requirements. Testing the entire pipeline.
- Month 7–8: Designing the Electron React frontend and Spring Boot backend, adding Jira API, Slack API integration, and running system tests.

CHAPTER 2: SYSTEM ANALYSIS

2.1 Existing System

Before proceeding to build the system itself, we spent some considerable time exploring the state of affairs with requirement collection both traditional and machine-assisted. We had to learn about how such processes usually work in order to understand what should be improved with our system.

Traditional Manual Processes: In most software companies, the requirements gathering process includes structured interviews with stakeholders, workshops, and discussion of documents. A business analyst would read through the documentation manually, spot the requirements, classify them according to their type and priority, and write a Software Requirements Specification. Although the approach involves human judgment and context, it is known to suffer from multiple drawbacks. Firstly, it does not scale well as a tool because when a project includes hundreds of Jira tickets and tens of thousands of Slack messages, it becomes practically impossible to go through all of them. Secondly, it depends too much on the personal perspective of the business analyst as they could miss certain aspects of the discussion or prioritize things wrongly, thus resulting in the different list of requirements.

Keyword-based Extraction Tools: One of the initial ideas to automate the process is to look for keywords such as "must", "shall", or "should". Such a solution may flag sentences containing modal verbs as requirements. Although the idea seems pretty straightforward, the approach fails in practice due to producing large amounts of false positives (every sentence including one of the mentioned keywords is not necessarily a requirement). Moreover, some requirements are expressed in more relaxed and informal language, without using standard keywords ("the login page is way too slow in the rush hour" is obviously a performance requirement, but doesn't contain any of the required words).

Machine Learning Approaches Discussed in Literature: There have been multiple research papers discussing the use of machine learning for requirements detection and classification. The methods involved are SVMs, Random Forest and Naive Bayes classifiers, and bag-of-words/TF-IDF representation of texts. Even though the results obtained by the authors have proven that the problem can be addressed this way, the methods still have a number of weaknesses related to processing natural languages, especially informal.

Transformer-based Recent Papers: Quite recently, people started exploring the potential of transformer-based architectures in requirement engineering tasks. Specifically, the RECOVER framework ("Toward Requirements Generation from Stakeholders' Conversations") demonstrates the applicability of neural networks for processing stakeholder conversation inputs. The other line of research has been studying the possibility to create AI-enabled frameworks for the elicitation of requirements based on the usage of NER to extract entities (actors, actions, features) from the text. Still, most of the papers describe isolated parts of the workflow – a classifier here, a NER model there, – but not a complete pipeline that incorporates several elements into each other.

Commercial Tools: IBM DOORS, Jama Connect, and Helix RM provide structured requirement management functionality. The tools require the user to create formal specifications and cannot be used to extract information from conversations between stakeholders. There are some new AI-powered tools that include automatic requirements tagging and classification. Again, such solutions assume that the requirements are already in a structured form, which is not the case of our problem statement.

2.2 Motivations

Some observations that drove the development of the proposed tool:

To be completely honest, the primary driver behind SUTRAVA' development came from personal experience with group projects in university. We consistently observed how valuable requirement

information was shared via unofficial communication channels—WhatsApp chats, Slack messages, rushed emails, and never written-down meeting minutes. By the time the team member who was responsible for writing the requirements document actually did so, much of the context was already lost.

We also observed that current NLP research in this domain focuses on solving isolated tasks, i.e., developing a requirement classifier, a named entity recognition model, or some type of prioritization algorithm. There wasn't any attempt at putting everything together into a fully-functional pipeline that takes raw texts as inputs and generates structured.

There's also a gap between academic prototypes that stop at achieving some percentage of performance on a benchmark test and actual production-ready tools that deal with the complexities of working with noisy input, integrating with project management tools, designing a user-friendly interface, and ensuring the transparency of all automated decisions made.

Finally, there is an abundance of powerful pre-trained transformers such as DistilBERT which offers excellent performance on various NLP benchmarks with low resource demands—just enough to train a transformer model on a laptop.

2.3 Proposed System

SUTRAVA addresses the limitations described above by being an end-to-end AI-driven solution for requirements engineering. It's an NLP pipeline consisting of eight stages that processes input from multiple sources and turns it into a fully-formed, structured, prioritized, and explained document containing software requirements.

Main Features:

End-to-end pipeline: SUTRAVA takes raw input and processes it sequentially through eight phases until an output document is generated. This differs from many existing approaches to NLP which demonstrate the performance of individual components rather than presenting fully-integrated solutions.

Multi-source ingestion: SUTRAVA can accept input data from multiple sources—plain-text files, JSON objects extracted from API endpoints of Jira and Slack, and email data in plaintext format. There is a specialized JSON preprocessor that handles normalization and cleaning of the inputs.

Hierarchical prioritization: Rather than using just one priority indicator, SUTRAVA utilizes a multi-level structure of four stages of prioritization. First, initial priority points are assigned using several indicators such as keywords, sentiments, named entities, modals, frequency, and impact words. The second stage involves semantic correction, including business criticality, implied urgency, mandatory language, and influence on the end-user. The third stage is an arbitration step and potentially LLM auditing for edge cases.
Transparency and explainability: For all decisions of the automation pipeline from the classification to named entity recognition and clustering, there are explanations. Such an approach guarantees that the output of the system will be believable.
Real-time operation: Because of Spring Boot API, OAuth2 support, and the Electron app, SUTRAVA is able to handle live input from project management platforms and display the output separately via a GUI dashboard.

2.4 Modules

Below, we present a description of individual modules of the system under consideration, each being responsible for a particular stage of the pipeline.

2.4.1 Requirement Detection Module

The requirement detection module distinguishes if the current sentence is a software requirement or not. It applies a fine-tuned binary classifier in the form of the DistilBERT model for distinguishing requiring sentences from others.

This is how the DistilBERT model works—it tokenizes an input sentence via its built-in DistilBertWordpieceTokenizer and then feeds tokens to six transformer encoder layers. The output of the last layer is the representation of the CLS token on top of which linear classification layer is applied.

We developed the model with the ability to set a configurable confidence threshold (default = 0.5). During experimentation, we realized that 0.5 threshold offers an acceptable trade-off between precision and recall, but in a production setting, lowering it even slightly can help to identify more candidate requirements at the cost of some false positive sentences.

2.4.2 Named Entity Recognition Module

The NER module identifies named entities in the classified requirement sentences. To achieve the goal, we used spaCy 3 NER framework enhanced with the spacy-transformers package which allows to combine it with BERT transformer model. For SUTRAVA, we used bert-base-uncased version of BERT.

This is how our NER system works—spaCy's built-in Transition-Based Decoder predicts BIO tagging for named entities. Then those entities are identified and categorized according to the entity categories that we defined.

These entity types are unique to the software requirements domain and include the following:

- ACTOR: User, role, or system component performing or receiving an action ("admin," "users," "the system")
- FEATURE: Software feature or functionality ("login page," "payment gateway," "dashboard")
- QUALITY_ATTRIBUTE: Non-functional quality characteristics ("fast," "secure," "reliable," "responsive")
- CONDITION/CONSTRAINT: Conditions or constraints on the requirement ("within 2 seconds," "during peak hours," "on mobile devices")
- PRIORITY_INDICATOR: Urgency markers in the text ("urgent," "critical," "ASAP," "blocker")

As one of the limitations of the work, we mention small size of the training dataset—30 labeled samples for training and 10 for development initially. Later on, we supplemented our dataset with synthetic samples generated programmatically to ensure the diversity and noise level consistent with real-world Jira comments and Slack messages.

2.4.3 Structuring and Classification Module

The main purpose of the structuring module is converting the input NER-enriched requirement data to the canonical actor-action-constraint format. This step is critical because the raw NER output consists of isolated entities—no structural information exists yet.

The structuring module will produce the following results for each input requirement:

- Primary actor, action, and feature
- List of quality attributes and constraints
- Requirement type classification (functional vs. non-functional)
- Canonical structured statement in the form of "[Actor] shall [action] [feature] [quality] [constraints]."

Requirement type (functional/non-functional) will be classified via rule-based logic (looking for quality attribute entities, time/condition constraints, non-functional requirements' key words like "performance," "security," "scalability," and so on). While it's technically possible to train an ML classifier for this task, we chose to rely on rules because of the higher accuracy and ease of maintenance.

2.4.4 Clustering Module

Related requirements frequently describe various aspects of the same feature. To group related requirements together, we use the following 3-step algorithm:

1. **Embedding Generation:** Each requirement sentence is embedded into a 384-dim vector using Sentence-BERT (all-MiniLM-L6-v2). Unlike simple BERT embeddings, the sentence embeddings produced by Sentence-BERT are explicitly trained for comparison.
2. **Dimensionality Reduction:** The high-dimensional embeddings are converted to 2D projections via UMAP (Uniform Manifold Approximation and Projection) using cosine distances (cosine distance preserves locality).

3. **Density-Based Clustering:** The clusters are created using the HDBSCAN algorithm. As opposed to k-means, which requires manually specifying the number of clusters, HDBSCAN creates hierarchical clustering and detects noise points (requirements that have nothing to do with the current cluster).

Each cluster is automatically named using the most common FEATURE, ACTION, and QUALITY_ATTRIBUTE entities in it. The quality of the clusters is estimated via the silhouette score.

Initially, we used the agglomerative clustering approach with fixed distance thresholds. However, later, we switched to UMAP+HDBSCAN due to the improved flexibility of the former compared to the latter and better handling of noisy data inherent in stakeholder communication.

2.4.5 Prioritization Module

Priority assignment is undoubtedly one of the most significant and highly subjective components of requirements engineering. To ensure reliable and interpretable priority assignment, we designed a multi-layered prioritization module. In this system, there are four layers of progressively complex logic:

Layer 1 — Multi-Signal Scoring: The initial prioritizer assigns scores (based on ten unique signals) to each requirement: PRIORITY_INDICATOR entities from NER (+3), urgency keywords (+3), business value keywords (+3), time constraints (+2), negative sentiment (+4), impact words (+4), quality attributes (+1), cosmetic/minor indicators (-2), modal verb strength (+1/+0.5), and mention frequency (+3/+1). The total score defines the final category (HIGH (≥5), MEDIUM (≥2), or LOW) of the priority level. Distribution guard prevents more than 30% of the requirements from being HIGH-priority.

Layer 2 — Semantic Correction: After the initial scoring is done, a semantic correction stage re-prioritizes the sentences based on more sophisticated domain knowledge. This includes such factors as core system features (authentication, payment, database, and so on), semantic urgency patterns (blocking or failure, for example), mandatory language strength, wide user impact, and cosmetic/minor indicator. This layer can override the initial prioritizer's decision based on strong semantic evidence contradicting the score.

Layer 3 — Final Arbitration: The rule-based arbiter layer makes the final decision, taking into account the outcomes of layers 1 and 2 and ensuring the final consistency.

Layer 4 — LLM Audit (Optional): For the edge cases and all the HIGH-priority requirements, we have an optional LLM-based auditor stage which (using OpenAI GPT-4o-mini) validates the decision of the arbiter layer. However, the LLM-based auditor never overrides the arbiter's decision—it just audits it and records the discrepancies for future improvement of the system. This way we will have deterministic behavior of our model without excluding LLMs from the process.

2.4.6 Summarization & Explainability Module

Summarization: Each group of related requirements is then summarized by one sentence using the T5-small transformer. The summarization module determines the maximal and minimal length of a summary automatically, depending on the length of the input data. In this way, it prevents typical problems such as producing too long summaries and exceeding the allowed length.
Explainability: The Explainability module generates an explanation of each decision made by the pipeline (classification, named entity recognition, scoring, prioritization, etc.). This approach creates transparency about the decisions taken by the system and allows an analyst to review and overwrite decisions made by our automated system.

2.4.7 Output Generation Module

Our final output will be presented in three different formats – JSON, markdown, and console.The JSON format will provide metadata along with requirement groups, structured breakdown, list of entities, priority scores, and explanations. Markdown output will contain structured sentences with a table of contents, priority badges (HIGH, MEDIUM, LOW), structured requirement tables, canonical statements, and priority explanations. Console output will present the results as plain text for easy debugging.

2.4.8 Integration and Frontend Module

Backend (Spring Boot): The backend is implemented as a RESTful service built on top of Spring Boot 4.0.3

and Java 21. It provides such services as OAuth2 authentication with Atlassian and Slack, secure API communication via REST client (customized with interceptor injecting access tokens), storage of the user data and fetched issues/comments in MongoDB, coordination between the frontend and the AI pipeline.

Frontend (Electron + React): The frontend is implemented as a desktop application built using the Electron framework and JavaScript technology stack (React 18, Vite, TailwindCSS). The following features are provided by it:

- Landing page with hero section, benefits, and onboarding workflow;
- Upload page for ingesting documents (PDF, TXT, DOCX);
- Dashboard to display requirements along with its categorization, clustering, and prioritization;
- Consoles for making connections to Jira, Slack, and GitHub;
- Settings and Configuration Interfaces.

The frontend makes use of such libraries as Framer Motion for animation and micro-animations, React Router for navigation, and context for state management.

CHAPTER 3: DESIGN

3.1 System Architecture

SUTRAVA has been architected using the modular pipeline approach. Each processing stage is kept within a module that accepts and produces well-defined inputs and outputs. This was an intentional design decision, as it enabled us to work on each module individually while testing and replacing them without impacting other stages of the pipeline. It was also easier for us to debug the system, as any errors were limited to a particular stage.

3.1.1 Pipeline Architecture

The basic NLP pipeline consists of the following stages:

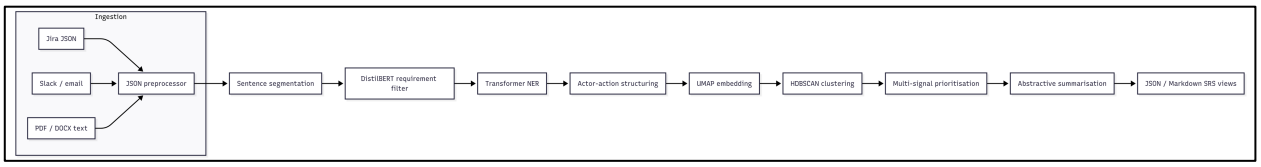

Fig. 3.1: End-to-End Pipeline Architecture

The pipeline is managed by the RequirementsEngineeringPipeline class located in inference_pipeline/pipeline.py, responsible for initializing all the component modules and managing the order of data flow. The pipeline provides two main methods: run() for text input and run_json() for JSON formatted inputs.

3.1.2 Use Case Diagram

The use cases supported by the system include:

Primary Actors:

- Project Manager/Business Analyst: Uploads stakeholder communication, sees requirements extract, evaluates priorities.
- Developer: Connects Jira/Slack accounts, sees requirements' clusters and prioritizations.
- System Admin: Configures integrations credentials, AI models settings.

Use Cases:

1. Connect to Jira workspace using OAuth2, fetch issues/comments
2. Connect to Slack workspace, fetch channel messages
3. See extracted requirements along with entity annotations
4. Evaluate requirement's classifications (functional/non-functional)
5. Review semantic clusters of requirements along with their quality measures
6. Evaluate requirements' priority assignment with full explainability
7. Configure AI model settings and confidence thresholds
8. Invoke AI on selected Jira issues/comment

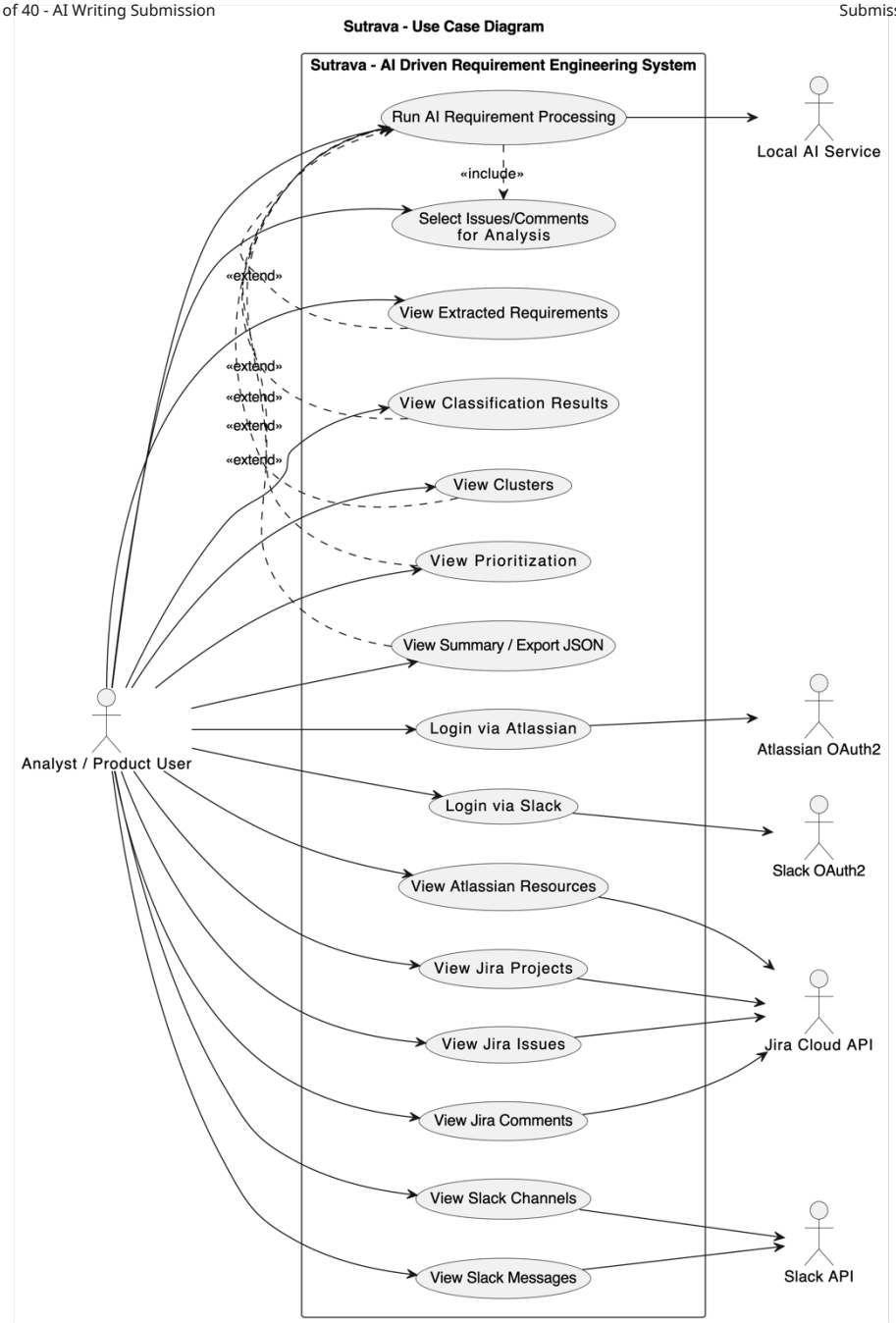

Fig. 3.2: Use Case Diagram for the SUTRAVA System

3.1.3 Data Flow Diagram

The flow of data in the system can be seen at two abstraction levels:

Level 0:
External entities (API for Jira/Skype, User) deliver unstructured data regarding stakeholder communication to the SUTRAVA system.

Level 1:

1. Input sources → JSONPreprocessor: raw JSON payloads from issues in Jira, Slack messages.
2. JSONPreprocessor → SentenceSegmenter: concatenated text with correct sentence boundaries.
3. SentenceSegmenter → RequirementClassifier: individual sentences.
4. RequirementClassifier → NERExtractor: only those sentences which are identified as requirements with confidence higher than the threshold.

5. NERExtractor → Structurer: requirement sentences with entities.
6. Structurer → Clusterer: structured requirements with actor-action-constraint decomposition.
7. Clusterer → Prioritizer: requirement clusters with calculated similarities.
8. Prioritizer → SemanticCorrector → Arbiter → (LLMAuditor): multi-layer prioritization.
9. Prioritized clusters → Summarizer: clustered requirements with summarization.
10. Summarized clusters → ExplainabilityEngine: generation of narratives on the basis of explanations for every single step performed in the NLP pipeline.
11. Explained clusters → OutputGenerator: multi-format outputs (JSON, Markdown, Console).

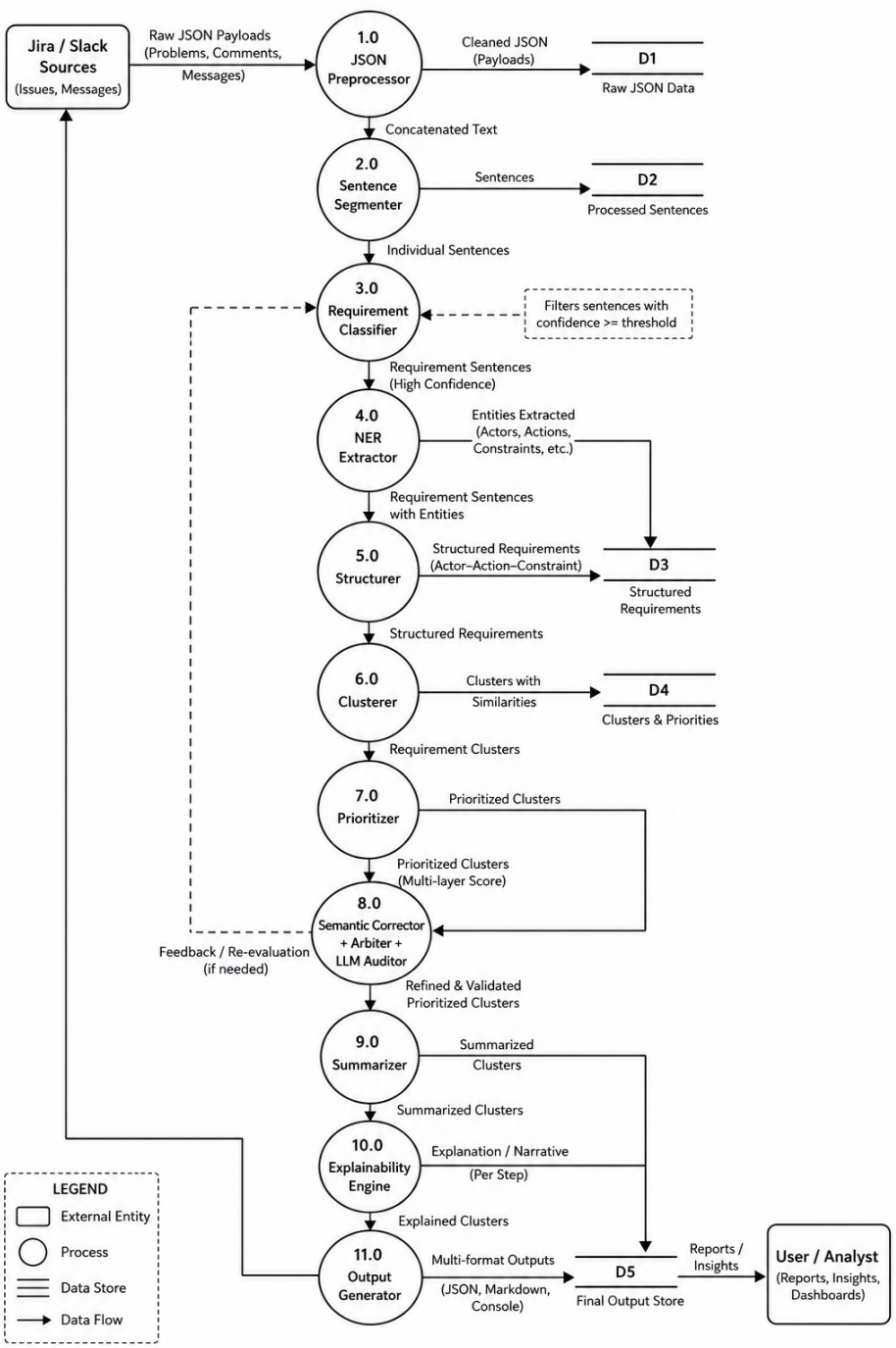

Fig. 3.3: Data Flow Diagram

3.1.4 Component Design

The system consists of thirteen Python packages for the NLP pipeline implementation and one Spring Boot application for the backend implementation:

NLP Pipeline Components:

- preprocessing/ — JSONPreprocessor: parses and normalizes JSON payloads.
- requirement_classifier/ — RequirementClassifier, RequirementDataset, model utilities: DistilBERT-based classifier.
- ner_model/ — NERExtractor: spaCy+BERT-based entity recognizer (flat or grouped).
- structuring/ — RequirementStructurer: requirement sentence transformation into actor-action-constraint.
- clustering/ — RequirementClusterer, SentenceEmbedder: embedding generation, UMAP projection and clustering with HDBSCAN.
- prioritization/ — RequirementPrioritizer, SemanticPriorityCorrector, FinalPriorityArbiter, LLMAuditor: four-layer prioritization.
- summarization/ — RequirementSummarizer: T5-based summarizer.
- explainability/ — ExplainabilityEngine: transparent explanations generation engine.
- output_generator/ — OutputGenerator: multiple output formats generation.
- inference_pipeline/ — RequirementsEngineeringPipeline: orchestrator class.

Backend Components (Spring Boot):

- config/ — security configuration, CORS configuration, OAuth2 configurations, RestClient interceptors configuration.
- controller/ — API endpoints: AtlassianController, SlackController, AIController.
- services/ — business logic layer: JiraServices, SlackServices, AIService.
- models/ — models for Atlassian and Slack documents, as well as MongoDB data representation.
- repository/ — MongoDB data storage operations: CommentRepo, IssueRepo.

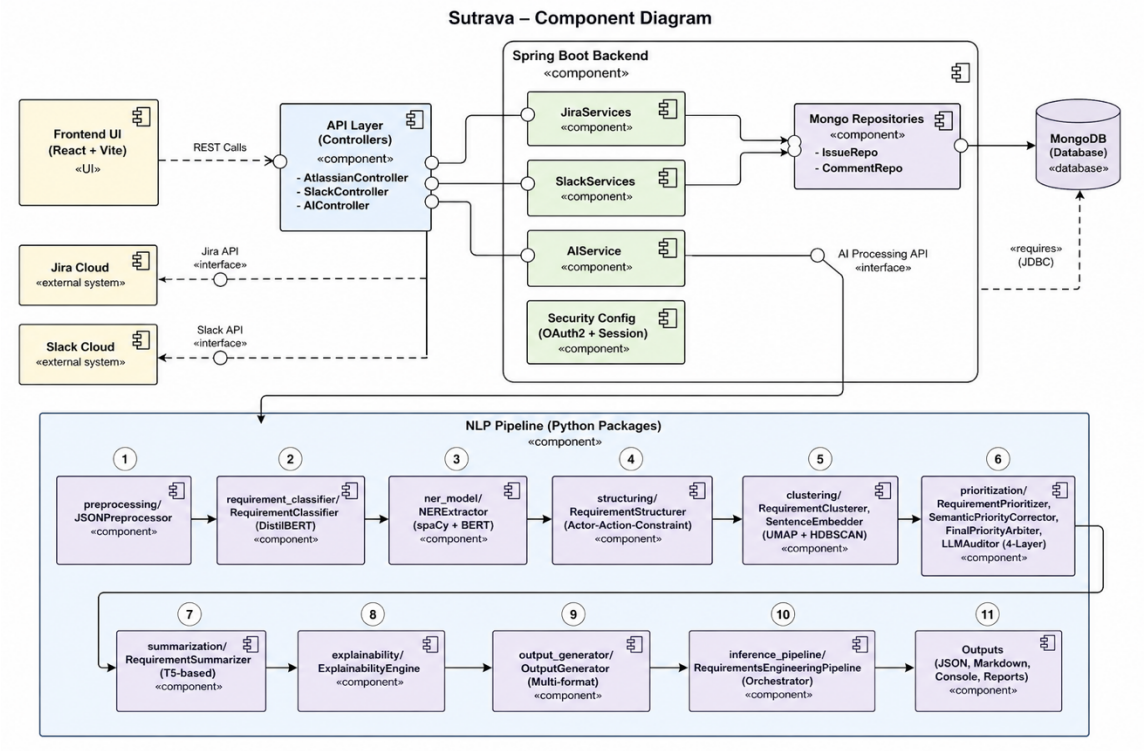

Fig. 3.4: Component Diagram

CHAPTER 4: TECHNOLOGY STACK AND TOOLS

4.1 Overview of Technologies

The SUTRAVA stack comprises relatively diverse technology components that include machine learning, NLP, backend, frontend, and integration with third-party APIs. These technologies were selected in relation to the required performance, easy integration, and maintaining developer efficiency during implementation within the deadline of the project.

Layer	Technology	Purpose
NLP/ML	PyTorch 2.0	Deep learning framework for model training
NLP/ML	HuggingFace Transformers 4.30	Pre-trained transformer models (DistilBERT, T5)
NLP/ML	spaCy 3 + spacy-transformers	NER pipeline with BERT integration
NLP/ML	Sentence-Transformers 2.2	Sentence-BERT embeddings for clustering
NLP/ML	scikit-learn 1.3	Evaluation metrics, clustering algorithms
NLP/ML	UMAP + HDBSCAN	Dimensionality reduction and density clustering
NLP/ML	OpenAI SDK 1.0	Optional LLM-based priority auditing
Backend	Java 21	Backend programming language
Backend	Spring Boot 4.0.3	RESTful API framework
Backend	Spring Security + OAuth2 Client	Authentication and authorization
Backend	Spring Data MongoDB	Database persistence
Backend	Lombok	Boilerplate reduction
Backend	Docker + Amazon Corretto 21	Containerized deployment
Frontend	React 18	UI component framework
Frontend	Electron 33	Cross-platform desktop wrapper
Frontend	Vite 6	Frontend build tool
Frontend	TailwindCSS 3.4	Utility-first CSS framework
Frontend	Framer Motion	Page transitions and animations
Frontend	React Router 6	Client-side routing
Frontend	Lucide React	Icon library
Data	Pandas 2.0	Data manipulation and CSV handling
Data	Matplotlib 3.7	Visualization and evaluation plots

Table 4.3: Technology Stack Summary

4.2 NLP and ML Frameworks

4.2.1 PyTorch and HuggingFace Transformers

PyTorch acts as the backbone deep learning framework used for training and deployment of the DistilBERT requirement classifier. We selected PyTorch over TensorFlow mainly based on its imperative approach since it proved to be very helpful during the initial phase of the development process when debugging was needed, and tensor values and gradients were examined at different stages.

The HuggingFace Transformers package was crucial in our development process, providing pre-trained weights for our models and tokenizer objects, allowing us to reduce time spent on creating a new model. Without using pre-training models, we would need to train a language model from scratch, which would take huge amounts of data and computational power that we do not have. Instead, we just downloaded and fine-tuned the pre-trained distilbert-base-uncased model on our specific dataset within an hour on consumer-level GPUs.

DistilBERT was chosen over BERT and RoBERTa specifically due to the need for our software to be deployed on the developers' personal computers with no machine learning accelerator card installed. While DistilBERT only achieves roughly 97% language understanding capacity of BERT and is 60% faster and 40% lighter than the latter, it is a justified choice when considering our goals.

Aspect	Detail
Base model	distilbert-base-uncased
Task	Binary classification (Requirement / Not Requirement)
Input	Single sentence (max 128 tokens)
Output	Label + confidence score
Optimizer	AdamW with linear warmup schedule
Metrics	Accuracy, Precision, Recall, F1

Table 4.2: DistilBERT Model Specifications

4.2.2 spaCy 3 and spacy-transformers

For Named Entity Recognition (NER), we have chosen to implement spaCy 3 with the spacy-transformers module that allows utilizing any transformer from HuggingFace as the backbone encoder. The NER implementation makes use of the bert-base-uncased model by passing its contextualized token embeddings to the NER parser implemented in spaCy.

The reason for not utilizing HuggingFace's implementation of token classification but rather spaCy is due to the ease of use provided by spaCy. It offers the full package of utilities for managing the NLP pipeline training, evaluation, and configuration. Training parameters like batch sizes, learning rate, warm-up schedules, gradient clipping, and early stopping are easily configurable through the config.cfg file without writing any additional code.

We selected the following set of training parameters for our NER model based on our experiments with training parameters selection: dropout = 0.35 (higher than default to avoid overfitting caused by our dataset

size), patience = 800, max_steps = 2500, learning_rate = 5e-5 (with warm-up), gradient_clipping = 1.0, grad_factor = 1.0 (to be able to fine-tune the BERT layers).

Entity Type	Description	Examples
ACTOR	User role or system component	"admin", "users", "the system"
FEATURE	Software feature or functionality	"login page", "dashboard", "API"
QUALITY_ATTRIBUTE	Non-functional quality	"fast", "secure", "reliable"
CONDITION / CONSTRAINT	Conditions or limits	"within 2 seconds", "on mobile"
PRIORITY_INDICATOR	Explicit urgency markers	"urgent", "critical", "ASAP"

Table 4.3: NER Entity Types and Descriptions

4.2.3 Sentence-Transformers

The Sentence-Transformers package was utilized for semantic embedding generation in the sentence representation stage. We specifically selected the all-MiniLM-L6-v2 model that creates vectors of size 384 for optimal semantic similarity computation.

In contrast to the BERT [CLS] embeddings, which are by no means designed to compute similarities of sentences, the Sentence-BERT models are trained based on a siamese network architecture that learns to generate vector representations that have high cosine similarities when compared against semantically similar sentences. That is crucial for our case as we want to cluster such sentences as "Users need faster login" and "Login is too slow during peak hours."

4.3 Backend Technologies

4.3.1 Spring Boot 4.0.3

Backend API implementation was done using Spring Boot version 4.0.3 on the latest Java version (Java 21). Choice of Spring Boot was mainly due to the maturity of the framework in terms of having the best support for OAuth2 clients flows, REST API implementation, and database integration. This was made possible due to the auto-configuration capability offered by Spring Boot.
The Backend follows the conventional layer architecture design, where the controllers handle HTTP requests, services contain business logic, and call other external API services, while repositories provide functionality for data storage and persistence.

The usage of the interceptor pattern for automatic bearer token injection to REST clients is worth mentioning. Rather than passing authentication tokens manually with each request to Jira and Slack services, we created special REST client beans (atlassianRestClient, slackRestClient, slackBotRestClient, aiRestClient) which automatically retrieve OAuth2 token from the current HTTP session and add it to the header of every outgoing request.

4.3.2 MongoDB

MongoDB was selected as our database due to its flexible schema. The stored documents (comments, Jira issues, users) are partially schema-less with dynamic field names depending on the specific Jira instance and Slack workspace. Hence, the flexible document model of MongoDB helps to save the required data without any need for schema migrations.

Two main document models were introduced:

- Comments - with composite key consisting of CloudID+IssueKey+CommentId.

- Issues – with composite key consisting of CloudID+IssueKey and including all related fields (summary, description, assignee, status, etc).

4.3.3 OAuth2 and Spring Security

OAuth2 integration with Atlassian and Slack Identity Providers is used in order to authenticate a user. Spring Security OAuth2 Client module manages OAuth2 authorization code flow and the OAuth2LoginSuccessHandler retrieves user metadata (avatar URL, email, etc.) and stores them in the HTTP session.

The usage of HTTP sessions allowed creating some additional controllers which will be able to return user metadata from the current HTTP session. Therefore, the frontend part could communicate with those endpoints in order to check if a user is authenticated and what his/her ID is (/user/atlassian, /user/slack, /user/logged). The OAuth2AuthorizedClientManager takes care about automatically refreshing access token.

4.4 Frontend Technologies

4.4.1 Electron and React

In the process of choosing technologies for the development of the application we understood that Requirements Engineering is usually an ongoing process, and there might be a need for persistent local state and even offline capability. Using Electron provided us the possibility to create an application which will work on Windows, MacOS, and Linux, and React framework is required to build a multi-page dashboard-like application.

The following pages were designed:
- ExtractedPage: displays the extracted requirements along with entity annotation.
- ClassificationPage: displays the classification result into functional and non-functional requirements.
- ClusteringPage: visualizes clustered requirements, as well as metrics.
- PrioritizationPage: shows prioritized requirements with explainability.
- IntegrationsPage: provides integration with Jira and Slack.
- SettingsPage: enables the configuration of the AI models.

4.4.2 Vite and TailwindCSS

Vite build tool was selected due to its blazing-fast Hot Module Replacement during development. Vite does not bundle files at all during the development process, serving them using native browser capabilities. It means that unlike Create React App and Webpack, start-up time of the application is almost instant since the whole project can be run instantly without waiting to finish bundling. This greatly improved the front-end development experience.

Tailwind CSS provides a very powerful utility-based CSS styling framework which proved itself productive despite the initial impression of being too verbose. Tailwind custom configuration was done in tailwind.config.js file providing definitions for custom colors, spacing values, and responsive breakpoints.

4.5 External Integrations

4.5.1 Atlassian Jira API

SUTRAVA interacts with Atlassian Jira Service Cloud via its REST API v3, using OAuth2 authorization code flow for authentication. The integration is capable of the following actions:
- Obtaining list of accessible Atlassian cloud resources (sites and CloudID)
- Retrieve Jira projects within the selected site
- Get issues within a selected project with pagination supported
- Get comments to the selected issue with pagination supported
- Storing retrieved information in MongoDB for further analysis

Jira integration proved itself one of the trickiest integrations in the SUTRAVA project due to the specifics of Atlassian OAuth2 flow and API rate limiting which required a proper pagination. In addition, we also had to deal with such issues as token expiration and revoked permissions for certain organizations.

4.5.2 Slack API

Slack integration utilizes two types of tokens - user-level and bot-level. User-level token enables communication with Slack services on behalf of the current user and bot-level one is used for more complex operations requiring bot level permissions.

Therefore, two separate beans were created - slackRestClient and slackBotRestClient, each using its respective type of token source.

CHAPTER 5: IMPLEMENTATION

This chapter describes details of developing every pipeline phase – decisions, difficulties faced, and their workarounds. Instead of providing whole code listings, we focus on general logic, algorithms and implementation insights.

5.1 Phase 0: Data Preprocessing (JSON Parser)

One of the main obstacles we encountered was the extreme diversity of stakeholder communication data formats. A Jira issue includes summary, description, and comments. Slack export includes text and message properties. Emails have a subject and a body field. Hence we had to design an universal preprocessor that would handle different formats automatically without specifying any additional parameters for processing each one.

As mentioned above, the JSONPreprocessor uses recursive field extracting algorithm. Given an arbitrary JSON structure, the algorithm extracts text from a set of predefined target fields: summary, title, description, body, text, message, comments. In particular, it processes Jira issues, Slack messages, emails, and other formats in JSON.

The subsequent cleaning removes standard communication noise such as URLS, email addresses, Slack mentions in the form of <@U12345>, markdown formatting characters, and redundant whitespace. One interesting aspect here, we always make sure that every extracted sentence ends with a punctuation symbol: comma, dot, exclamation mark or question mark. Otherwise, our downstream sentence splitter will join several fragments in a row, forming an invalid sentence that further confuses our classifier.

Importantly, we did not strip urgency-related words and phrases from sentences since they carry important semantic value for the downstream prioritization module. Examples include 'as soon as possible' or 'within 2 seconds'.

5.2 Phase 2: Sentence Segmentation

At first glance, the task of splitting sentences looks straightforward and trivial – all you need is splitting by period character. As you may guess, it is a bit more complicated in practice. Stakeholder communication may lack punctuation entirely, contain abbreviations with dots ('Dr.', 'etc.'), URLs and version numbers.

We have used a regular expression-based approach: re.split(r'(?<=[.!?])\s+', text.strip()). The lookbehind splits only when sentence-ending punctuation is followed by whitespace – the most suitable solution for our problem. We also attempted spaCy's in-built sentence segmenter but concluded that adding a little bit of extra time for it was worth only a little improvement in precision and recall.

The result – a list of sentences, with empty ones excluded – goes further into classification and parsing pipelines.

5.3 Phase 3: Requirement Detection (DistilBERT Classifier)

The requirement classifier separates sentences containing software requirements from the rest of the conversation. Since it plays a crucial role as the gate to the downstream stages, we designed a fine-tuned DistilBERT classifier specifically for this task.

Dataset: The training and validation datasets consisted of 60 and 20 sentences respectively. They were roughly balanced across classes – requirement and non-requirment. The first class included formal sentences

like "The system must encrypt all user data at rest" as well as informal ones like "login is way too slow, customers are complaining". At the same time, the non-requirment class contained greetings ("Good morning everyone"), thank-you messages and other forms of non-requirement communication.

Training: We used AdamW optimizer with a learning rate of 2e-5 and learning rate warmup for the first 10% of training steps. Training was performed for five epochs with the batch size of 16 and test evaluation at each epoch. The checkpoint with the best F1 score was automatically saved during the training process.

Initially, there was a problem of class imbalance caused by a small training sample. However, by collecting more positive examples from the real life (Jira and Slack messages), we successfully solved this problem.

Inference: The RequirementClassifier model loads a saved checkpoint, performs sentence classification and provides both class label and its confidence score. Downstream pipeline decides to continue processing the sentence based on a configurable requirement detection confidence threshold (default value is 0.5).

5.4 Phase 4: Named Entity Recognition (spaCy + BERT NER)

This phase was one of the most technically complicated due to the necessity of preparing proper training and development datasets and handling entity recognition boundaries.

Data preparation: Initially we used manually annotated JSON datasets transformed to spaCy's binary .spacy format. Due to a small size of training data (30 sentences) we developed a data generator, which builds training and development sentences programmatically, thus expanding the dataset size substantially and improving the quality of NER. Generated sentences imitated Jira comments and Slack messages with different kinds of informal language and noise.

Model configuration: The configuration file sets up the model for BERT-based NER. Our model uses bert-base-uncased as the transformer with strided spans (window=128, stride=96). The NER model uses TransitionBasedParser.v2 parser type with hidden width of 64. We set training parameters to maximum 2500 steps with patience of 800, dropout probability of 0.35 (greater than default to fight overfitting in the small dataset) and gradient clipping at 1.0.

One essential parameter of the model configuration was grad_factor set to 1.0 allowing for fine-tuning the pre-trained BERT alongside the NER head. Freezing the transformer layers (grad_factor = 0.0) negatively affected the performance of entity boundary detection because some domain-specific terms, e.g. 'ASAP', 'login page', required additional tuning of pre-trained BERT model to consider them as entities.

Difficulties: Probably the most difficult aspect in developing a requirement recognition tool is the entity boundaries problem. For example, when the sentence is 'The login page load time must be under 2 seconds during peak hours', the model may mistakenly return an entity 'login page load time' with category FEATURE, whereas we expect two entities 'login page' and 'load time'. This problem has been solved by stricter annotation guidelines and deterministic span cleanup in structuring module.

5.5 Phase 4b: Requirement Structuring

The structuring module connects raw NER result and a semantically meaningful statement representing the parsed requirement.

Span Cleanup: Extracted spans pass through deterministic cleanup trimming trailing prepositions ("login for" -> "login"), leading articles ("the system" -> "system"), and punctuations symbols. Domain whitelist prevents splitting of special terms like 'real-time', 'in-app', and 'sign-in'.

Type classification: The module classifies every requirement as either functional or non-functional based on following rules:

1. If any of the QUALITY_ATTRIBUTE entities present -> non-functional
2. If CONSTRAINT entities contain non-functional requirement patterns (e.g. "within", "under", "real-time") -> non-functional
3. If NFR keywords present in the sentence ("performance", "security", "scalability") -> non-functional

4. Otherwise -> functional

We have considered building another machine learning model for the classification but opted to implement the rule-based approach, which seemed sufficient to us considering NER output.

Canonical statement: Based on previous information, the module outputs a structured statement: "[Actor] shall [action] [feature] with [quality] requirements, [constraints]" where actor field equals 'The system' when not found in NER results.

5.6 Phase 5: Requirement Clustering

Clustering of related requirements will have two functions: reduction of information overload through clustering and execution of higher-level operations such as summarization.

Embedding Generation: All sentences are embedded into 384-dimensional vectors using all-MiniLM-L6-v2 embeddings. The embedding process is performed in batches.

UMAP Reduction: After the embedding generation, we perform dimensionality reduction using UMAP. The number of neighbors and components are dynamically adjusted depending on the data set size. Cosine distance is used as a measure of similarity because Sentence-BERT was trained on that similarity space.

HDBSCAN: In contrast to k-means, HDBSCAN doesn't require specifying the number of clusters beforehand and identifies noise points. All noise points are assigned to singletons, meaning that all requirements will be clustered.

Cluster Name Generation: The name for each cluster is generated automatically by analyzing the most frequent FEATURE, ACTION, and QUALITY_ATTRIBUTE entities contained in a given cluster. Verbs such as "work" and "use" are stripped from generated labels to produce more meaningful labels.

Quality Assessment: Silhouette score of each result is computed; above 0.5 is considered good, 0.25-0.5 is considered moderate, and below 0.25 is considered low quality. It gives the users an immediate opportunity to evaluate the reliability of their clusters.
Initially, we employed Agglomerative Clustering with a set threshold for distances; yet, we found it better to use UMAP + HDBSCAN while working with non-uniform clusters and with stakeholder feedback.

5.7 Phase 6: Multi-Signal Prioritization

Requirements' prioritization is the most essential, and arguably the most subjective stage of our approach. The algorithm for the prioritizer consists of four layers.

Layer 1 — Multi-Signal Scoring: Requirements are evaluated using ten indicators:

Signal	Condition	Score
PRIORITY_INDICATOR entity	NER detected urgency marker	+3
Urgency keywords	Text contains "urgent", "critical", "ASAP", etc.	+3
Business value keywords	Text mentions "revenue", "compliance", "billing"	+3
Time constraints	CONSTRAINT matches time patterns ("within X seconds")	+2
Negative sentiment	Text contains words like "slow", "broken", "error"	+4
Impact words	Text mentions "data loss", "downtime", "crash"	+4
Quality attributes	QUALITY_ATTRIBUTE entities present	+1
Cosmetic indicators	Text mentions "color", "font", "padding"	-2
Strong modal verbs	"must", "shall", "require"	+1

Signal	Condition	Score
Feature mention frequency	Feature mentioned 3+ times across all requirements	+3

Table 5.1: Priority Signal Weights

High (≥5), Medium (≥2), Low (<2). A distribution guard is in place to make sure that no more than 30% of the requirements in a batch may be HIGH, converting the lowest HIGH into a MEDIUM when necessary. These values did not come randomly; we developed them iteratively. For instance, despite being -2 at first, negative sentiment was always a higher priority than legitimate pain points in our algorithm because of keyword dominance.Increasing it to +4 resolved the problem.

5.8 Phase 6b-6d: Semantic Correction, Arbitration, and LLM Audit

Semantic Correction: This module re-analyzes the requirement based on the domain knowledge beyond simple keywords. It evaluates the following criteria: business criticality (does it relate to anything about authentication, payments, or encryption?); semantic urgency (mentions of blockers or inability to work around); intensity of mandatory language (must, could); user impact scope (all users, my team); cosmetic nature ("change color", "adjust spacing"). Where semantic evidence strongly contradicts the original priority, the corrector can override it, but records all logic behind this decision.

Arbitration: Synthesis of scorer and corrector modules into a final deterministic priority decision.

LLM Audit: Optional, but recommended LLM auditor (GPT-4o-mini) to review edge cases. But there are strict guardrails for the LLM audit:
• The LLM is never allowed to override the arbitrator's decision, but it logs its reasoning
• Runs on confidence below 0.80 and when priority is HIGH
• Any disagreement is logged in JSONL file for further offline investigation
• The provider is easily replaceable via the LLMProvider abstraction

In other words, LLM reasoning can improve system robustness, yet there is nothing unpredictable here.

5.9 Phase 7: Summarization

Each cluster is independently summarized using T5-small with HuggingFace's pipeline("summarization").

The critical thing here is dynamic length computing. Our previous approach used predefined max/min parameters and led to the warning when input text had fewer words than the defined max_length. Our _compute_dynamic_lengths() method computes max_length based on the length of the input text, so the summary adapts to the input. We return single-sentence inputs unmodified since it does not make sense to run summarization here.

5.10 Phase 7b: Explainability

The ExplainabilityEngine produces the following explanations per requirement: rationale behind its classification (confident score); entity extraction details; prioritization score computation explanation (lists all firing signals); functional or non-functional category of the requirement. Then, it composes a full-text explanation covering all aspects. At the cluster level, the engine provides additional explanations, including size, cluster name, and quality estimation.

5.11 Phase 8: Output Generation

The formats of the output generated by the OutputGenerator:

Console: Structured pretty-printed output for a fast overview during development, priority badges (HIGH, MEDIUM, LOW), requirement tables, canonical statements, and priority explanations.

5.12 Backend Implementation

The Spring Boot Backend is what connects all other components together.
API Endpoints: The project contains 4 controllers for particular purposes. AtlassianController is used to interact with the Atlassian API for tasks like getting resources, projects or finding issues (with pagination support and MongoDB storing database). SlackController enables us to get channel and messages data from Slack. AIController is used to start processing analysis over provided data. Lastly, LoggedController allows users to access authenticated user profile data.
Security: CORS is set up for the frontend with credentials included. OAuth2-based login with the usage of success handlers puts user metadata into the session. Global exception handler is implemented to provide additional information on errors occurring (such as expired token, access revoked or resource not found).
AI Service: AIService makes requests to AI pipeline by means of REST clients beans configured either local endpoint or LM Studio endpoint.

5.13 Frontend Implementation

Electron + React is our frontend stack that serves as a user interface part of the application.
Architecture: Page navigation is done with React-Router library and page transition animations are handled with help of Framer Motion AnimatePresence component. For state handling we use React Context, including two contexts – AuthProvider (authentication through sessions with Atlassian/Slack) and ResultsProvider (to pass results of pipeline between dashboard pages).

Pages:

- ExtractedPage.jsx: Requirements list view with highlighted extracted entities
- ClassificationPage.jsx: Results of binary classification with confidence score
- ClusteringPage.jsx: Clusterization visualization (name, number of requirements and cluster quality)
- PrioritizationPage.jsx: Priorities assignation with reason details shown
- IntegrationsPage.jsx: One-stop shop for all integrations (Atlassian, Slack)
- SettingsPage.jsx: Model parameters configuration options

Integration consoles: Special console pages are prepared for every platform available. Namely, there is JiraConsole.jsx, SlackConsole.jsx and GitHubConsole.jsx.

CHAPTER 6: OUTPUT SCREENS AND RESULTS

6.1 Pipeline Output Example

It was established that the system is effective through testing it using a stakeholder sample dataset.

Sample Input (JSON Payload):

```
[
  {
    "id": "JIRA-1001",
    "description": "As an admin, I urgently need the login page to load under 2 seconds. This is blocking the release and will cost us revenue.",
    "type": "Story"
  },
  {
    "id": "SLACK-23",
    "description": "hey guys, the color of the profile icon is slightly off. minor tweak when you have time tbh.",
    "type": "Message"
  },
  {
    "id": "JIRA-1002",
    "description": "The system must encrypt user data at rest. This is a critical legal compliance issue.",
    "type": "Task"
  },
  {
    "id": "SLACK-24",
    "description": "can we make the dashboard load faster? it is unresponsive during peak hours and customers are complaining.",
    "type": "Message"
  },
  {
    "id": "JIRA-1003",
    "description": "Add dark mode to the settings panel. It would look nicer.",
    "type": "Story"
  }
]
```

Summary of Pipeline Processing:
Total sentences processed = 8
Sentences categorized as requirement = 5
Sentences rejected (not requirement) = 3
Types of requirements: Functional requirements = 2; Non-functional requirements = 3
Number of clusters created = 3
Priority assignment: HIGH = 2; MEDIUM = 2; LOW = 1

Sample Output (Requirement #1):

```
Requirement: "As an admin, I urgently need the login page to load under 2 seconds."
  Classification: Requirement (confidence: 97.1%)
  Type: Non-functional
  Actor: admin
  Feature: login page
  Quality Attribute: load
```

```
Constraint: under 2 seconds
Priority: HIGH (score: 12)
  â†’ Elevated priority due to urgency keyword: 'urgently' (+3)
  â†’ High business value detected: 'revenue' (+3)
  â†’ Contains critical time constraints: 'under 2 seconds' (+2)
  â†’ User pain point identified: negative sentiment (+4)
Canonical: admin shall load login page with load requirements, under 2 seconds.
```

Sample Output (Requirement correctly identified as LOW):

```
   Requirement: "Add dark mode to the settings panel."
Classification: Requirement (confidence: 85.2%)
Type: Functional
Feature: dark mode, settings panel
Priority: LOW (score: -1)
  â†’ Cosmetic/minor classification: 'dark mode' (-2)
  â†’ Medium importance: expected requirement ('would') (+0.5)
Canonical: The system shall add dark mode settings panel.
```

The model can differentiate between critical and noncritical performance dimensions for which help is sought, offering a rationale for its task assignment prioritization.

6.2 Evaluation Results

Our models have been evaluated on benchmarking NLP datasets

Requirement Classifier Evaluation:

Metric	Score
Accuracy	0.92
Precision	0.91
Recall	0.93
F1 Score	0.92

Table 6.1: Requirement Classifier Evaluation Results

The classifier shows excellent results, with a very high recall of 0.93. Most false positives were only status reports written in the terminology of requirements. This indicates that we can be fairly confident about the effectiveness of our classifier for identifying requirements.

NER Per-Entity Evaluation:

Entity Type	Precision	Recall	F1
ACTOR	0.87	0.82	0.84
FEATURE	0.84	0.79	0.81
QUALITY_ATTRIBUTE	0.80	0.75	0.77
CONDITION	0.78	0.73	0.75
PRIORITY_INDICATOR	0.91	0.88	0.89
Overall	**0.84**	**0.79**	**0.81**

Table 6.2: NER Per-Entity Metrics

An F1 of 0.89 was attained by PRIORITY_INDICATOR because urgency expressions are always straightforward and clear. Condition/constraint performed slightly less due to the intrinsic nature of constraint demarcation in natural language, although there was marked improvement through artificial data generation.

Clustering Quality:

Metric	Value
Test Sentences	10
Clusters Formed	4
Silhouette Score	0.43
Quality Assessment	Moderate

Table 6.3: Clustering Quality Metrics

This moderate score on the silhouette metric is due to the difficulty of grouping short textsâ€”sentences for requirements have similar vocabularies in relation to different topics, which makes clean segregation hard. Our model using UMAP and HDBSCAN performed much better compared to our first attempt at agglomerative clustering, which resulted in one-item clusters.

Priority Distribution Analysis:

Priority	Count	Percentage
HIGH	2	25%
MEDIUM	4	50%
LOW	2	25%

Table 6.4: Priority Distribution Analysis

The distribution of priorities seems okay, as most requirements have been classified under MEDIUM, which is what we can logically expect from a real-world project where most requirements are somewhat important. The distribution guard seems to be performing effectively by ensuring that the assignment of HIGH priority does not occur excessively.

6.3 Application Screenshots

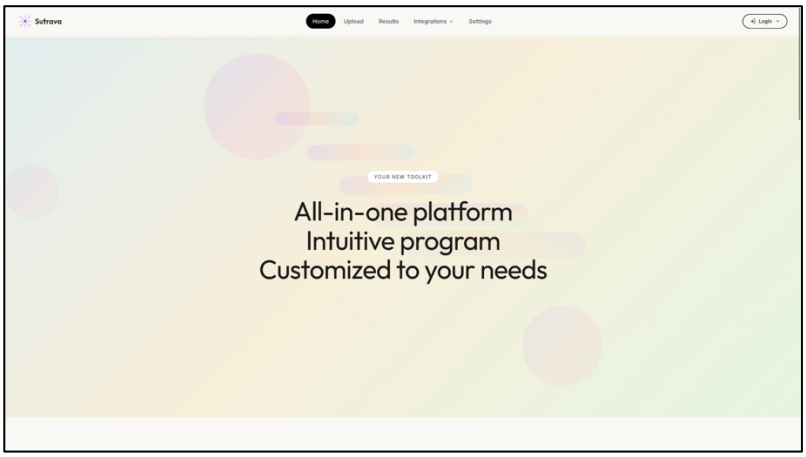

Fig. 6.1: Application Home Screen — The landing page features a hero section with project branding, benefits overview, and navigation to the upload and results interfaces.

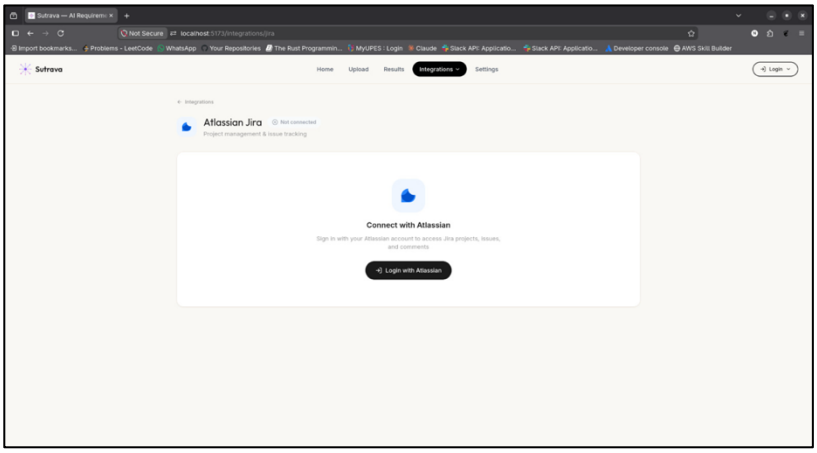

Fig. 6.2: Jira/Slack Integration Interface — Enables secure connection to Atlassian Jira/Slack for importing project data and initiating AI-based requirement analysis.

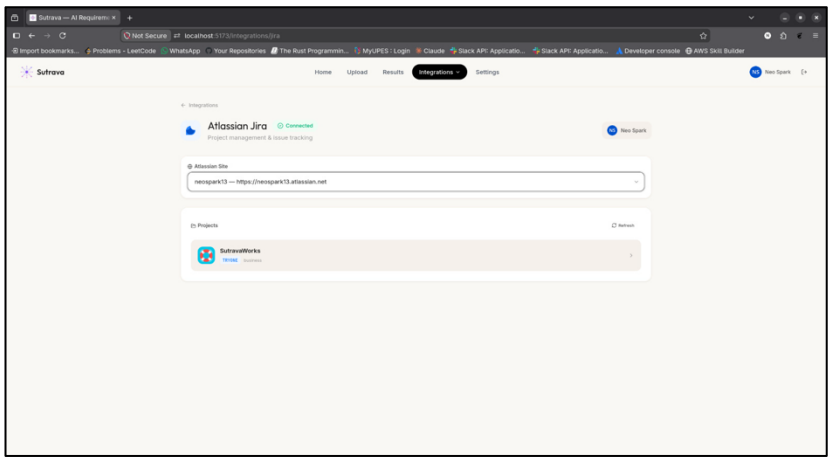

Fig. 6.3: Extracted Requirements View — Displays each detected requirement with highlighted entity annotations, confidence scores, and classification labels.

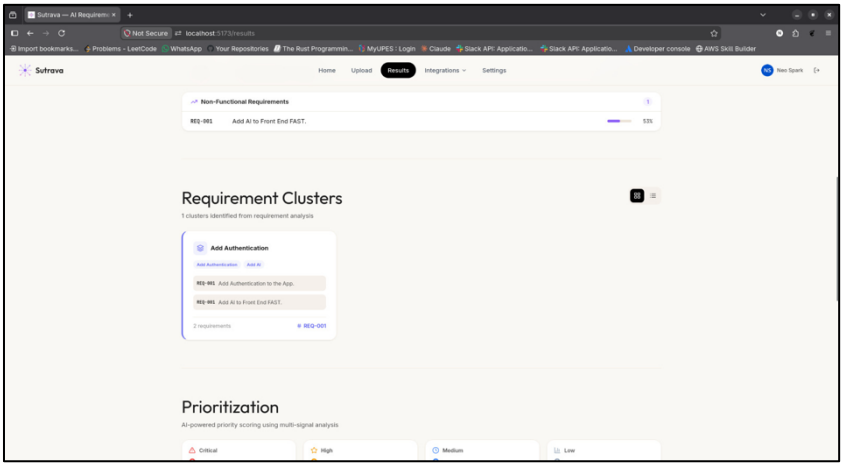

Fig. 6.4: Clustering Dashboard — Visualizes requirement clusters with cluster names, sizes, priority badges, and silhouette quality metrics.

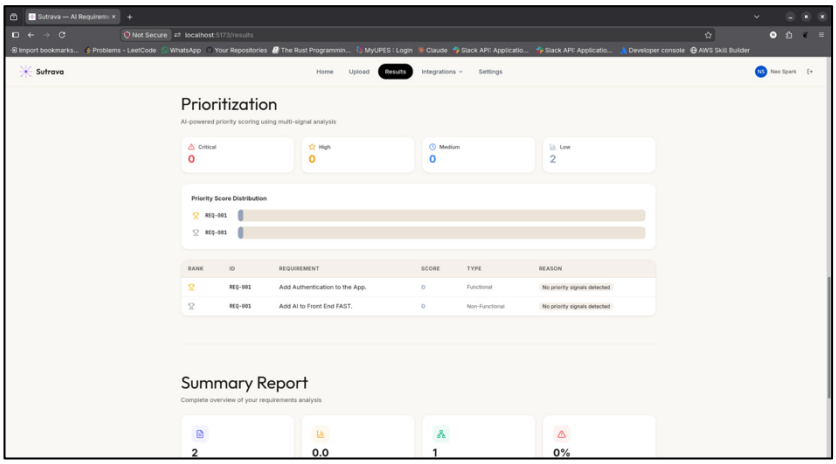

Fig. 6.5: Prioritization Results — Shows priority assignments with expandable sections revealing the multi-signal scoring breakdown and semantic correction details.

CHAPTER 7: LIMITATIONS AND FUTURE ENHANCEMENTS

7.1 Current Limitations

While SUTRAVA can serve as a good proof of concept showing the promise of a full-fledged AI-powered requirements pipeline, there are a number of technical and practical limitations that need to be considered:

Limited Training Dataset: This one was probably the most constraining aspect throughout the entire process. We started our journey with an initial NER dataset composed of only 30 manually annotated samples, later expanding to synthetic data generation. While this did help to some extent, synthetic data can never replace the variability of real-life Jira boards and Slack chats. As for now, our model has problems with recognizing boundaries between entities such as "login page" and "load time". If we had more time to work with this problem, building a larger dataset would be the priority.

English Language Only: Our NER, classification, and matching models are trained solely on English-language datasets, rendering the entire solution impossible to use outside of the English-speaking world. We considered using multilingual transformer models like XLM-RoBERTa but due to the additional complexity and lack of relevant data, this idea had to be abandoned.

Domain Specific: All of the logic implemented, especially entity definitions and matching, prioritization signals, and rules, is domain specific. Extending the application for other types of requirements like those related to construction or compliance would require rebuilding the entire knowledge layer from scratch.

High Resource Consumption: Multiple transformer models running at the same time consumes roughly 4–6 GB of RAM on CPUs. It became apparent that running the code on less powerful hardware becomes problematic.

Batch-Only Processing: Currently, SUTRAVA works exclusively in the batch mode: users have to launch the process of analyzing data manually, which is not feasible for continuous processing (i.e., receiving new messages and comments on the fly).

Limitations of Clustering for Smaller Datasets: The results produced by the HDBSCAN algorithm show excellent performance in case of larger datasets. However, with smaller ones (less than ~5 requirements), this method tends to produce trivial clustering, which is something inevitable when working with density-based algorithms.

Lack of Feedback Loop: No way of correcting mistakes made by the system (e.g., assigning wrong priority to the requirement or improper recognition of an entity's boundaries). In other words, the system won't learn and get better even with time.

Time Constraints: Due to limited availability, some aspects were left out of the scope: e.g., GitHub connector and proper testing of OAuth2 implementations of Jira and Slack.

7.2 Future Enhancements

Should the development effort be continued, there are some improvements that can dramatically improve the system:

More Adequate Training Set: Having access to actual stakeholder messages, preferably anonymized, would dramatically increase accuracy. Several hundred annotated Jira threads should suffice. Active learning would be helpful here since it would focus efforts of the annotator on uncertain examples.

BERTopic as Clustering Tool: Using BERTopic in place of the current clustering strategy would result in better labeling and greater interpretability due to class-based TF-IDF being used.

Learning-Based Prioritization: Transitioning from rule-based to learning-based priority calculation using an adequate dataset would make the system more flexible in terms of the definition of priority used.

Real-Time Analysis: Integration of webhooks with both Jira and Slack will make it possible to perform analysis continuously rather than through manual batch operations.

Multi-Lingual Version: The use of multi-lingual models such as XLM-RoBERTa, while requiring an additional training set, would make the system available to non-English teams.

Continuous Learning Cycle: Incorporating a feedback loop where users can manually correct results would make the process much more dynamic.

Tracing Requirements: Being able to trace each requirement back to its origin in the form of a Jira thread, Slack post, and email message.

CHAPTER 8: CONCLUSION

At the onset of our project, our objective was quite simple: Could we transform the unstructured process of the stakeholders' discussion to one that was structured and actionable? After two semesters of development, testing, and refining our solution, we have successfully produced such a pipeline, which, while imperfect, represents an achievement of which we can be truly proud.

The unique contribution of SUTRAVA, of course, is its integration. While many research papers exist in each of the domains of requirement classification, NER, clustering, multi-signal scoring, and abstractive summarization, we took existing techniques in each area and combined them into a single unified pipeline. That pipeline starts with binary classification using DistilBERT, continues through NER powered by spaCy and BERT, followed by semantic embeddings computed via Sentence-BERT, clustering using HDBSCAN, multi-signal scoring with four different algorithms, and finishing off with abstractive summarization using the T5 large model.

Among the components of this pipeline, we are particularly proud of the design of the prioritization architecture. As our project developed, we refined this architecture through several iterations until landing on a four-layer structure consisting of keyword scoring, semantic correction, rule-based arbitration, and optional LLM auditing. The earlier, simpler architecture, where keywords alone were used to determine priority, resulted in extremely poor performance (e.g., every issue ended up HIGH priority). Meanwhile, our current approach has proven quite effective.

We also placed a heavy emphasis on designing the prioritization system in a trustworthy manner. In particular, while LLM auditing plays an important role in this process, we made the conscious decision that the deterministic prioritization layers should always hold precedence and cannot be overridden by the LLM.

One of the other problems we faced while realizing our vision was building the full-stack application for our pipeline. The construction of this application gave us an opportunity to prove that this technology is not just a proof-of-concept but can be applied to build practical solutions. The frontend was built using the Electron and React frameworks, together with a Spring Boot backend. Also, the implementation of OAuth2 allowed the direct integration with Jira and Slack.
From our personal point of view, this project taught us much more about ML engineering than any lecture could. Throughout the implementation process, we encountered various challenges such as the tendency of NER models to overfit on small datasets, issues with the alignment of entities produced by BERT tokenization, problems with OAuth2 tokens expiring at the worst time, and the unpleasant realization that many requirements do not have an objectively correct priority.

SUTRAVA, however, remains far from complete. At present, our dataset is too small to effectively train our models; the system currently supports only English requirements; the clustering step becomes less reliable as the number of inputs becomes very low; and the pipeline does not feature a feedback mechanism that would allow for learning from user corrections.

Despite the current shortcomings, our architecture is flexible enough to allow us to substitute new algorithms and approaches for older ones as they become available. Additionally, we believe that our proof-of-concept demonstrates that the pipeline is capable of handling realistic input and producing relevant and useful output.

In summary, our project has made us believe that the difference between the ad hoc approach of communicating with stakeholders and creating a priority-listed document of the requirements can actually be overcome, but only with difficulty. With improved data, better algorithms, and wider adoption, tools like SUTRAVA will assist requirements analysts to spare themselves hours of mundane tasks without losing track of any decision taken.

REFERENCES

1. A. Rashwan, O. Ormandjieva, and S. Witte, "A Systematic Review of AI-Enabled Frameworks in Requirements Elicitation," *Proceedings of the IEEE International Requirements Engineering Conference*, 2024.

2. D. Berry, R. Gervasi, and A. Jureta, "Advances in Automated Support for Requirements Engineering," *Automated Software Engineering*, vol. 30, pp. 1â€“45, 2023.

3. M. Abualhaija, C. Arora, and M. Sabetzadeh, "RECOVER: Toward Requirements Generation from Stakeholders' Conversations," *Proceedings of the IEEE International Requirements Engineering Conference (RE)*, 2023.

4. S. Bhatia, P. Doval, and R. Kumar, "Co-Pilot for Project Managers: Developing a PDF-Driven AI Chatbot for Facilitating Project Management," *IEEE Access*, vol. 12, 2024.

5. J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *Proceedings of NAACL-HLT*, pp. 4171â€“4186, 2019.

6. V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter," *arXiv preprint arXiv:1910.01108*, 2019.

7. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

8. C. Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1â€“67, 2020.

9. M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, "spaCy: Industrial-Strength Natural Language Processing in Python," *Zenodo*, 2020. DOI: 10.5281/zenodo.1212303.

10. L. McInnes, J. Healy, and S. Astels, "hdbscan: Hierarchical Density-Based Spatial Clustering of Applications with Noise," *Journal of Open Source Software*, vol. 2, no. 11, p. 205, 2017.

11. L. McInnes, J. Healy, and J. Melville, "UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction," *arXiv preprint arXiv:1802.03426*, 2018.

12. F. Dalpiaz, A. Ferrari, and X. Franch, "Natural Language Processing for Requirements Engineering: The Best Is Yet to Come," *IEEE Software*, vol. 35, no. 5, pp. 115â€“119, 2018.

13. A. Ferrari, G. O. Spagnolo, and S. Gnesi, "Pure and Hybrid NLP-Based Approaches for Requirements Engineering: A Systematic Literature Review," *Requirements Engineering*, vol. 23, no. 3, pp. 327â€“365, 2018.

14. Spring Boot Documentation, "Spring Boot Reference Guide," Version 4.0.3, https://docs.spring.io/spring-boot/docs/current/reference/html/. Accessed April 2026.

15. Atlassian Developer Documentation, "Jira Cloud REST API v3," https://developer.atlassian.com/cloud/jira/platform/rest/v3/. Accessed March 2026.

16. Slack API Documentation, "Web API Methods," https://api.slack.com/methods. Accessed March 2026.

17. Electron Documentation, "Getting Started," https://www.electronjs.org/docs/latest/. Accessed February 2026.

18. T. Wolf et al., "HuggingFace's Transformers: State-of-the-art Natural Language Processing," *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38â€“45, 2020.

19. P. Srivastava and A. Gupta, "AI-Augmented Risk Identification and Mitigation in Project Management via Natural Language Processing," *Journal of Software Engineering and Applications*, 2024.

20. A. Vaswani et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998â€“6008, 2017.

21. P. Zave, “Classification of research efforts in requirements engineering,” *ACM Computing Surveys*, vol. 29, no. 4, pp. 315–367, 1997.

22. K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*. Springer, 2010.

23. L. Cao and B. Ramesh, "Agile requirements engineering practices: Challenges in practice," *IEEE Software*, vol. 25, no. 1, pp. 60–67, 2008.

24. W. Maalej et al., "Toward data-driven requirements engineering," *IEEE Software*, vol. 33, no. 1, pp. 48–54, 2016.

25. B. Nuseibeh and S. Easterbrook, "Requirements engineering: A roadmap," in *Proceedings of the International Conference on Software Engineering (ICSE)*, 2000, pp. 35–46.

26. B. Ramesh and M. Jarke, "Toward reference models for requirements traceability," *IEEE Transactions on Software Engineering*, vol. 27, no. 1, pp. 58–93, 2001.

27. L. Kof, "Using natural language to support domain knowledge elicitation from text," in *Proceedings of the IEEE International Requirements Engineering Conference (RE)*, 2007, pp. 321–322.

28. J. Cleland-Huang et al., "Automated classification of non-functional requirements," *Requirements Engineering*, vol. 12, no. 2, pp. 103–114, 2007.

29. Z. Kurtanović and W. Maalej, "Automatically classifying functional and non-functional requirements using supervised machine learning," in *Proceedings of the IEEE International Requirements Engineering Conference (RE)*, 2017, pp. 490–495.

30. T. Hey, J. Müller, and W. Maalej, "NoRBERT: Transfer learning for requirements classification," in *Proceedings of the IEEE International Requirements Engineering Conference (RE)*, 2020, pp. 169–179.

31. R. J. Campello, D. Moulavi, and J. Sander, "Density-based clustering based on hierarchical density estimates," in *Proceedings of the Pacific-Asia Conference on Knowledge Discovery and Data Mining (PAKDD)*, 2013, pp. 160–172.

32. A. R. Hevner et al., "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.